Ders 4 Çalışma Özeti

Ders 4 Çalışma Özeti

Genel Konular

- Dersin ilk yarısı uygulama: Arş. Grv. Muhammet Taha Gökcan iki soru çözdü. İkinci yarıda Dr. Ahmet Elbir string ve matris konularıyla devam etti.
- Asistanın sesi kayıtta yer yer kesik ve anlaşılmaz. Kod ayrıntıları ekran görüntülerinden alındı.
- Soru 1: Pisagor üçlüsü
  - Verilen bir dizide $a^2 + b^2 = c^2$ eşitliğini sağlayan bir üçlü olup olmadığını bulup ekrana yazdıran program. Örnek dizi: 3, 13, 5, 6, 4, 12, 8.
  - Fikir: bütün üçlü kombinasyonları denemek. Seçmeli sıralamaya (selection sort) benzer şekilde indisler iç içe ilerler: `i`, `j = i+1`, `k = j+1`. Böylece aynı kombinasyon iki kez denenmez.
  - Üçlü sıralı gelmeyebilir (ör. 3, 5, 4). Bu yüzden üç durum birlikte kontrol edilir: $x + y = z$, $x + z = y$, $y + z = x$ ($x = a_i^2$ gibi).
  - Kod yapısı:
    - `#define MAX 10`, `int Array[MAX];` Dizi boyutu `scanf` ile kullanıcıdan alınır, elemanlar tek tek okunur: `scanf("%d", &Array[i]);`. `&` hücrenin adresini verir.
    - Üç iç içe `for`: `for(i=0; i<N; i++) for(j=i+1; j<N; j++) for(k=j+1; k<N; k++)`.
    - Koşul sağlanırsa sayaç `s` arttırılır ve üçlü yazdırılır.
  - Derste derleme sırasında satır sonunda `;` eksikliği hatası (`expected ';' before 'y'`) görüldü ve düzeltildi.
  - Tek satırlık `if`/`for` gövdesinde süslü parantez zorunlu değildir, ama birden fazla satır varsa gerekir.
- Soru 2: İki boyutlu konvolüsyon
  - Girdi (input) matrisinin üzerinde bir çekirdek (kernel) matrisi gezdirilir. Her konumda üst üste gelen elemanlar çarpılır, çarpımlar toplanır ve sonuç çıktı (output) matrisine yazılır.
  - Çekirdek önce sol üst köşeye oturur, sonra bir sağa kayar. Satır sonuna gelince bir alt satıra geçer.
  - Çekirdeğin ortasındaki elemanın girdideki pikseli temsil ettiği düşünülebilir. Bu yüzden çekirdek boyutu genelde tektir (3, 5…). Kenarlarda karşılık gelen piksel olmadığı için çıktı küçülür: 3×3 çekirdekte her kenardan bir satır/sütun kaybolur.
  - Çıktı boyutu: `output_size = input_size - kernel_size + 1`. 4×4 girdi ve 3×3 çekirdek için 2×2.
  - Girdi kontrolü `do-while` ile yapılır: hatalı değer girilirse tekrar sorulur.
    - `while(input_size > MAX || kernel_size % 2 == 0 || output_size < 1);`
    - `do-while` sayesinde okuma kodunu iki kez yazmak gerekmez, en az bir kez çalışır.
  - Hesaplama dört iç içe döngüdür:
    - `for(i=0; i<output_size; i++) for(j=0; j<output_size; j++) { sum = 0; for(ki=0; ki<kernel_size; ki++) for(kj=0; kj<kernel_size; kj++) sum += input[i+ki][j+kj] * kernel[ki][kj]; ... }`
    - Sonuç işaretçi aritmetiğiyle yazıldı: `*(*(output + i) + j) = sum;` (`output[i][j] = sum` ile aynı).
  - Hoca konvolüsyonun önemini anlattı: görüntüden kenar, köşe, ortalama renk gibi ayırt edici özellikler çıkarmak için kullanılır. Yapay zekâ ve makine öğrenmesinde, özellikle evrişimli sinir ağlarında (CNN) temel işlemdir. Tek boyutlu hâli de vardır.
- Konvolüsyon filtreleri (Dr. Ahmet Elbir)

  - Keskinleştirme (sharpen): merkezdeki pikselin 5 katından dört komşusu (üst, alt, sol, sağ) çıkarılır.
  - Köşegen kenar bulma: bir köşegene 1, diğer köşegene -1 verilir.
  - Ortalama/bulanıklaştırma: katsayılar 1, 2, 4 gibi seçilir ve katsayılar toplamına (16) bölünür. Gauss bulanıklaştırma çan eğrisi şeklinde katsayılar kullanır.
  - Birim (identity) çekirdek (ortada 1, çevresi 0) görüntüyü değiştirmez, sadece kenarlardan birer piksel kaybolur: 500×500 → 498×498.
  - Uygulama: göğüs röntgenlerinden (zatürre, COVID, normal) hastalık tahmini. Önce konvolüsyonlarla özellik çıkarılır, sonra sınıflandırma yapılır.
  - Gürültü türleri: tuz-biber (salt and pepper) gürültüsü. Farklı amaçlar için farklı filtreler (filtre bankaları) kullanılır.
- Örüntü eşleme (pattern matching)
  - Bir metinde (text) aranan bir örüntünün (pattern) hangi indisten başladığını bulmak. Word'deki Ctrl+F gibi.
  - `int indexOfV1(char *ptr1, char *ptr2)`:
    - `len1 = strlen(ptr1)`, `len2 = strlen(ptr2)`.
    - `for(i=0; i <= len1-len2; i++)`: örüntü metnin dışına taşmasın diye son başlangıç indisi `len1 - len2`'dir (kayan pencere, sliding window).
    - `while(*ptr1 == *ptr2 && matchCount != len2) { matchCount++; ptr1++; ptr2++; }`
    - `matchCount == len2` ise eşleşme bulunmuştur: `return i;`.
    - Değilse `ptr1 -= (matchCount - 1); ptr2 -= matchCount; matchCount = 0;` ile metinde bir sonraki başlangıca, örüntüde başa dönülür.
    - Bulunamazsa `return -1;`.
  - Fonksiyona dizi `char *` olarak gönderilebilir, çünkü dizi adı aslında ilk elemanın adresidir. Dizi olarak alınsa da içeride yapılan değişiklik dışarıya yansır.
  - Versiyon 2: `string.h` içindeki `strstr` fonksiyonu. Metinde örüntüyü bulursa adresini, bulamazsa `NULL` döndürür.
  - Çalışma sorusu (versiyon 3): eşleşme bozulunca başa dönmek yerine daha ileriden devam eden bir algoritma yazmak (önemli uyarı için "Ders sonrası notlar"a bakın).
- Matris içinde örüntü arama
  - `#define MAXROW 10`, `#define MAXCOLUMN 10`, `char text[MAXROW][MAXCOLUMN]`. Kullanıcı M×N boyutunu girer.
  - `void fillMatrix(char text[][MAXCOLUMN], int M, int N)`: her hücreye `(char)(rand() % 3 + 'A')` yazar. `rand() % 3` 0, 1 veya 2 verir, `'A'` (65) eklenince A, B veya C olur.
  - Matris fonksiyona gönderilirken satır sayısı yazılmayabilir, ama sütun sayısı mutlaka yazılmalıdır. Derleyici bir satırın bellekte ne kadar yer kapladığını bilmek zorundadır.
  - `scanf("%s", pattern)`: `pattern` zaten bir dizi (adres) olduğu için & kullanılmaz.
  - Örüntü uzunluğu: `while(pattern[len] != '\0') len++;`. `while(pattern[len++] != '\0');` yazılırsa son karşılaştırmada da arttırıldığı için uzunluk bir fazla çıkar.
  - `void search(char text[][MAXCOLUMN], int M, int N, char pattern[])`:
    - `bound = N - len;` her satırda en fazla bu indise kadar başlangıç denenir.
    - `for(r=0; r<M; r++) for(i=0; i<=bound; i++) { j=0; while(j<len && text[r][i+j]==pattern[j]) j++; if(j==len) { count++; printf(...); } }`
    - Hiç bulunamazsa "not found" yazılır.
  - Yapısal programlamada değişkenler blok başında tanımlanır, kodun ortasında rastgele tanımlanmaz.
- Çok boyutlu diziler ve işaretçiler
  - `arr[1][2]` ile `*(*(arr + 1) + 2)` aynıdır: `arr + 1` ikinci satıra gider, `*(arr+1)` o satırın başlangıç adresidir, `+ 2` iki eleman ilerler, dıştaki * değeri getirir.
  - İlk değer verme: `int exap[5][3] = {{1,2,3}, {4}, {5,6,7}};` her iç süslü parantez bir satırdır, eksik elemanlar 0 olur. `int exap[5][3] = {1,2,3,4,5,6,7};` ise elemanları üçer üçer satırlara yerleştirir: 1 2 3 / 4 5 6 / 7 0 0, kalan satırlar 0.
  - İşaretçi dizisi: `char *p[5];` beş işaretçiden oluşan bir dizidir. İşaretçinin işaretçisi: `int **r;`.

Hocanın Özellikle Vurguladığı Kısımlar

- Bütün kombinasyonları denerken indisleri bir öncekinden başlatın, aynı kombinasyonu tekrar denemeyin.
- Kullanıcı girdisini `do-while` ile kontrol edin.
- Matrisi fonksiyona gönderirken sütun boyutunu mutlaka yazın.
- Dizi zaten bir adres olduğu için `scanf("%s", dizi)` içinde `&` kullanılmaz.
- Değişkenleri blok başında tanımlayın.

Kısa Tekrar Notları

- Konvolüsyon çıktı boyutu: `input - kernel + 1`.
- `sum += input[i+ki][j+kj] * kernel[ki][kj]`.
- `arr[i][j]` ≡ `*(*(arr+i)+j)`.
- `rand() % 3 + 'A'` → A, B, C.
- `strstr(metin, oruntu)` bulamazsa `NULL`.
- `{1,2,3,4,5,6,7}` ile `[5][3]`: 1 2 3 / 4 5 6 / 7 0 0.

Detaylı Açıklamalar

Konvolüsyon mu, korelasyon mu?

- Derste ve kodda çekirdek ters çevrilmeden doğrudan çarpılıyor. Matematikteki konvolüsyonda çekirdek 180° döndürülür. Ters çevirmeden yapılan işlemin adı çapraz korelasyondur (cross-correlation). Derin öğrenme kütüphanelerinde bu işleme yine de "konvolüsyon" denir. Simetrik çekirdeklerde (ortalama, Gauss) iki işlem aynı sonucu verir.
- Hoca konvolüsyonu "ağırlıklı aritmetik ortalama, katsayılar toplamına bölünür" diye özetledi. Bu bulanıklaştırma gibi katsayıları pozitif olan filtreler için geçerlidir. Kenar bulma ve keskinleştirme çekirdeklerinde katsayıların toplamı 0 veya 1 olabilir, toplama bölünmez.

Ders sonrası notlar

- **Kodların yeniden kurulması:** Ekran görüntülerinden alınan kodlar yeniden yazılıp `gcc` ile çalıştırıldı. Pisagor programı örnek dizide iki üçlü bulur: (3, 5, 4) ve (13, 5, 12). Örüntü eşleme ve matris içinde arama kodları doğru çalışır. `fillMatrix` + `search` ile 5×5 matriste "BC" araması doğru konumları buldu. Örüntü sütun sayısından uzunsa `bound` negatif olur, döngü hiç çalışmaz ve "not found" yazılır.
- **Konvolüsyon slaydındaki çıktı:** Slayttaki 4×4 girdi (1-16) ve çekirdek `[[1,0,1],[0,-1,1],[-2,1,1]]` için çıktı 2, 2, 2, 2 olarak gösteriliyor. Kesin hesap `[[8, 10], [16, 18]]` verir. Sol üst eleman: $1 + 3 - 6 + 7 - 18 + 10 + 11 = 8$. Çekirdek katsayılarının toplamı 2 olduğu ve girdi doğrusal arttığı için çıktı elemanlarının hepsi eşit olamaz. Slayttaki değerler büyük olasılıkla temsilidir, kendi kodunuzu bu değerlerle sınamayın.
- **`printf` yerine `scanf`:** Ekranda bir süre `printf("%d", &kernel_size);` satırı görülüyor. Çekirdek boyutunu okumak için `scanf("%d", &kernel_size);` olmalıdır. `printf` değeri okumaz, `kernel_size` değişkeni rastgele değerle kalır.
- **Versiyon 3 önerisi hakkında uyarı:** Derste "eşleşmeyen karakterden sonrasına atlamak çok daha doğru olur" dendi. Bu basit atlama bazı durumlarda eşleşmeyi kaçırır. Örnek (derste yazılmadı, test edildi): metin "AAAB", örüntü "AAB". İndis 0'dan "AA" eşleşir, 'A' ile 'B' eşleşmez. Eşleşmeyen konumun ötesine atlanırsa indis 1'deki gerçek eşleşme kaçar (doğru cevap 1, atlamalı yöntem -1 verir). Güvenli bir şekilde ileri atlamak için örüntünün kendi içindeki tekrarları hesaba katan Knuth-Morris-Pratt (KMP) algoritması kullanılır.
- **Geri alma miktarı:** Derste bir yerde "match etmeyen karakter sayısı kadar" geri alınacağı söylendi (01:35:50). Kodda `ptr1`, eşleşen karakter sayısının bir eksiği kadar (`matchCount - 1`) geri alınır. Hiç eşleşme yoksa `ptr1` bir ileri gider, yani yine sonraki başlangıca gelinir (gcc ile denendi: `indexOfV1("ABCABAC", "ABAC")` 3, `indexOfV1("AAAB", "AAB")` 1 döndürdü).

- **`arr + 1` işaretçinin işaretçisi mi?** Derste `arr + 1` için "işaretçinin işaretçisi" dendi. Teknik olarak `int arr[3][3]` için `arr + 1` tipi `int (*)[3]`'tür: "3 elemanlı int dizisini gösteren işaretçi". `int **` değildir. Bu yüzden statik bir matris `int **` parametreli bir fonksiyona doğrudan gönderilemez. Denemede `sizeof(*(arr+1))` 12 bayt (bir satır), `sizeof(**(arr+1))` 4 bayt verdi.
- **`rand` ve `srand`:** `srand` çağrılmadan `rand` her çalıştırmada aynı sayı dizisini üretir. Farklı matrisler için `srand(time(NULL));` kullanılır (`<time.h>`).
- **`scanf("%s")` güvenliği:** `%s` boşlukta durur ve dizi boyutunu kontrol etmez. Uzun girişte dizi taşar. `char pattern[10]` için `scanf("%9s", pattern)` yazmak daha güvenlidir.
- **Belirsiz kalan yer:** Asistanın 00:54 civarındaki anlatımı (altyazıda "üçer, üçer…" tekrarı) ses kalitesi nedeniyle anlaşılamadı. O sırada ekranda girdi ve çekirdek matrislerini okuyan iç içe döngüler yazılıyordu. Anlatımın 3×3 çekirdekle ilgili olduğu tahmin ediliyor.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.